

Deferred Execution vs Immediate Execution in LINQ

In this article, I will discuss the difference between **LINQ Deferred Execution vs Immediate Execution in C#** with Examples. Please read our previous article discussing the [LINQ Zip Method in C#](#) with an Example. In LINQ, understanding the concepts of deferred execution and immediate execution is important, as they determine when the query is actually executed and the results are fetched.

Deferred Execution vs Immediate Execution in LINQ

The LINQ queries are executed in two different ways, as follows.

1. [LINQ Deferred Execution](#)
2. [LINQ Immediate Execution](#)

Based on the above two types of Execution, the LINQ operators or methods are divided into 2 categories. They are as follows:

1. **Deferred or Lazy Operators:** These Query Operators are used for Deferred Execution. For example – Select, SelectMany, Where, Take, Skip, etc. belong to the Deferred or Lazy Operators Category.
2. **Immediate or Greedy Operators:** These Query Operators are used for Immediate Execution. For example – Count, Average, Min, Max, First, Last, ToArray, ToList, etc., belong to the Immediate or Greedy Operators category.

LINQ Deferred Execution using C#:

Deferred execution is a concept in LINQ where the execution of a query or an operation is delayed until the actual results are needed. LINQ queries are composed of a sequence of operations, and some of these operations are executed only when the query results are explicitly requested.

When you create a LINQ query against a collection, database, or any other data source, it does not immediately execute. Instead, the query is represented as an expression tree. The actual execution occurs when you iterate over the results, such as by using methods like ToList(), ToArray(), FirstOrDefault(), or in a for-each loop.

Deferred execution can lead to more efficient queries, especially when dealing with large datasets, as it allows for optimization by the underlying data provider. Let us understand LINQ Deferred Execution with an example. The example below is self-explained. So, please go through the comment lines.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public int Salary { get; set; }
    }
}
```

```

class Program
{
    public static void Main()
    {
        List<Employee> listEmployees = new List<Employee>
        {
            new Employee { ID= 1001, Name = "Priyanka", Salary = 80000 },
            new Employee { ID= 1002, Name = "Anurag", Salary = 90000 },
            new Employee { ID= 1003, Name = "Preety", Salary = 80000 }
        };
        // In the below statement the LINQ Query is only defined and not executed
        // If the query is executed here, then the result should not display
        IEnumerable<Employee> result = from emp in listEmployees
                                      where emp.Salary == 80000
                                      select emp;
        // Adding a new employee with Salary = 80000 to the collection listEmployees
        listEmployees.Add(new Employee { ID = 1004, Name = "Santosh", Salary = 80000 });

        // The LINQ query is actually executed when we iterate thru using foreach
        foreach (Employee emp in result)
        {
            Console.WriteLine($" {emp.ID} {emp.Name} {emp.Salary}");
        }
        Console.ReadKey();
    }
}

```

Output:

```

1001 Priyanka 80000
1003 Preety 80000
1004 Santosh 80000

```

When to Use Deferred Execution in LINQ?

Deferred execution in LINQ means that the execution of a query is postponed until the query result is actually enumerated or evaluated. In LINQ, the decision to use deferred execution or immediate execution depends on the specific requirements and scenarios. Here are some considerations for when to use deferred execution in LINQ:

- **Performance Optimization:** If you have a complex query or a query that involves remote data sources, using deferred execution can optimize performance. The query is not executed until the results are actually needed, which can reduce unnecessary computations.
- **Lazy Loading:** Deferred execution supports lazy loading, where data is loaded only when it is requested. This is beneficial when dealing with large datasets, as you can avoid loading the entire dataset into memory at once.

- **Dynamic Query Composition:** Deferred execution allows you to compose queries based on user inputs or conditions dynamically. You can build up a query incrementally, and the execution is deferred until the final results are needed.
- **Querying External Data Sources:** When working with external data sources like databases or web services, using deferred execution can be advantageous. The query is translated into a form that is suitable for the external data source, and execution is delayed until the actual data is needed.
- **Chaining Operations:** Deferred execution allows you to chain multiple LINQ operations together without executing the query until the final results are requested. This can lead to more concise and readable code.

LINQ Immediate Execution using C#

In LINQ, queries are typically written to express the desired operations on data, but the execution of those queries is deferred until the results are actually needed. This deferred execution is a key feature of LINQ and provides benefits like optimization opportunities. However, there might be scenarios where you want to force immediate execution of a LINQ query. Immediate execution means that the query is executed as soon as it is defined, without waiting for further operations or iterations.

In immediate execution, the query is executed as soon as it is defined. When you use methods like `ToList()`, `ToArray()`, or `FirstOrDefault()`, it triggers the immediate execution of the query, and the results are retrieved right away.

Immediate execution is useful when you want to get the results immediately or if you want to ensure that the data is current at the time of execution. Let us see an example to understand LINQ Immediate Execution in C# with an example. The following example is self-explained. So, please go through the comment lines.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public int Salary { get; set; }
    }
    class Program
    {
        public static void Main()
        {
            List<Employee> listEmployees = new List<Employee>
            {
                new Employee { ID= 1001, Name = "Priyanka", Salary = 80000 },
                new Employee { ID= 1002, Name = "Anurag", Salary = 90000 },
                new Employee { ID= 1003, Name = "Preety", Salary = 80000 }
            };
            // In the following statement, the LINQ Query is executed immediately
            // Using the ToList() method which is a greedy operator which forces
            query
```

```

// to be executed immediately
IEnumerable<Employee> result = (from emp in listEmployees
                                where emp.Salary == 80000
                                select emp).ToList();

// Adding a new employee with Salary = 80000 to the collection listEmployees
// will not have any effect on the result as the query is already executed
listEmployees.Add(new Employee { ID = 1004, Name = "Santosh", Salary = 80000 });

// The above LINQ query is executed at the time of its creation.
// This is proved because Santosh is not included in the result
foreach (Employee emp in result)
{
    Console.WriteLine($" {emp.ID} {emp.Name} {emp.Salary}");
}
Console.ReadKey();
}
}
}

```

Output:

```

1001 Priyanka 80000
1003 Preety 80000

```

Example:

The following example uses Immediate Execution as we use the greedy operator, which returns a single value.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public int Salary { get; set; }
    }
    class Program
    {
        public static void Main()
        {
            List<Employee> listEmployees = new List<Employee>
            {
                new Employee { ID= 1001, Name = "Priyanka", Salary = 80000 },
            }
        }
    }
}

```

```

        new Employee { ID= 1002, Name = "Anurag", Salary = 90000 },
        new Employee { ID= 1003, Name = "Preety", Salary = 80000 }
    };
    // In the following statement, the LINQ Query is executed immediately
are
    // Using the Count() method which is a greedy operator which forces
    // to be executed immediately
    var result = (from emp in listEmployees
                  where emp.Salary == 80000
                  select emp).Count();
    // Adding a new employee with Salary = 80000 to the collection listEmployees
    // will not have any effect on the result as the query is already executed
    listEmployees.Add(new Employee { ID = 1004, Name = "Santosh", Salary = 80000 });
});

    // The LINQ query is executed at the time of its creation.
    // This is proved because Santosh is not included in the count

    Console.WriteLine($" Employees with Salary 80000 : {result}");
    Console.ReadKey();
}
}
}

```

Output:

```
Employees with Salary 80000 : 2
```

When to Use Immediate Execution in LINQ?

Immediate execution refers to the execution of the query at the point when it is defined. You might choose to use immediate execution in LINQ in the following scenarios:

- **Small Data Sets:** If you work with small data sets and the query is simple, immediate execution might be more efficient. The overhead of creating and managing deferred execution for small queries could be unnecessary.
- **Debugging:** Immediate execution can be beneficial during debugging. When you use immediate execution, the query is executed immediately, allowing you to inspect the results and debug any issues immediately. In deferred execution, the query is executed when the results are enumerated, which might make debugging more challenging.
- **Predictable Performance:** Immediate execution can provide more predictable performance in some cases. With deferred execution, the actual execution of the query might be delayed until the results are needed, making it harder to predict when the query will execute.
- **Database Operations:** When working with LINQ to SQL or LINQ to Entities in Entity Framework, some queries need to be executed immediately. For example, when you need to fetch a single value, such as using `First()`, `Single()`, or `Count()`, immediate execution is triggered to retrieve the result from the database.

When should you choose Immediate Execution and Deferred Execution in C#?

In LINQ, we have the option to use either immediate execution or deferred execution based on our specific requirements. The choice between immediate and deferred execution depends on the scenario and the nature of the LINQ query. Let's understand each concept:

Immediate Execution in C#:

- Immediate execution means that the query is executed as soon as it is defined. The results are calculated and returned immediately.
- Methods that trigger immediate execution include `ToList()`, `ToArray()`, `ToDictionary()`, `Count()`, `First()`, `Single()`, and others that force the query to be executed right away.
- Use immediate execution when you need the entire result set immediately and want to work with the concrete collection in memory.

Deferred Execution in C#:

- Deferred execution means that the query is not executed immediately when it is defined. Instead, it is executed only when the results are actually needed or when a method that triggers execution is called.
- Deferred execution allows for more efficient querying, especially when dealing with large datasets, as it postpones the execution until necessary.
- Methods that support deferred execution include `Where()`, `Select()`, `OrderBy()`, and other standard query operators.

Choosing Between Immediate and Deferred Execution in C#:

Use immediate execution when you need the results immediately, especially if you are working with small datasets or want to ensure the data is fetched and processed immediately.

Use deferred execution when you want to build a query but delay its execution until necessary. This is particularly beneficial for large datasets, as it allows for optimizations and reduces unnecessary processing.

In the next article, I will discuss the [ToList and ToArray Methods in C#](#) with Examples. In this article, I explain the difference between **LINQ Deferred Execution vs Immediate Execution using C#** with Examples. I hope you understood the LINQ Deferred Execution vs Immediate Execution using C#.